

Module 2: BVN Verification

Invalid BVN length

API Test Case Documentation

Module: BVN Verification

API: Initialize BVN Consent

Test Case ID: TC03

Test Scenario

Initialize BVN Consent with a BVN whose length is less than the required 11 digits.

Objective

To verify that the API correctly validates BVN length and returns an appropriate client error response when an invalid BVN is provided.

Preconditions

- A valid API token is generated and available
 - API endpoint is up and accessible
 - Required headers are configured properly
-

Test Data

Field	Value
BVN	22222222 (8 digits – invalid)

Steps to Execute

1. Set HTTP method to **POST**

2. Enter the Initialize BVN Consent endpoint URL
3. Add required headers:
 - Authorization: Bearer Token
 - Content-Type: application/json
4. Add the request payload:

```
{  
  "bvn": "22222222"  
}
```

5. Click **Send** to execute the request
-

Expected Result

- HTTP Status Code: **400 Bad Request** (or *422 Unprocessable Entity*)
- Response Body:

```
{  
  "status": "error",  
  "message": "BVN must be exactly 11 digits"  
}
```

- Error message should be clear, concise, and user-friendly
-

Actual Result

- HTTP Status Code: **500 Internal Server Error**
- Response Body:

```
{  
  "status": "error",  
  "message": "child \"bvn\" fails because [\"bvn\" length must be 11 characters long]"  
}
```

Result Status

Failed

Defect Summary

The API incorrectly returns a server error (500) for a client-side validation issue and exposes a technical error message that is not user-friendly.

Observations

1. Incorrect HTTP status code returned for invalid input
 2. Error message is too technical and exposes internal validation structure
 3. Lack of proper input validation handling at API level
 4. Response format is not standardized for validation errors
-

Impact Analysis

- Misleads API consumers into thinking there is a server failure
 - Poor developer experience due to unclear error messaging
 - May cause improper error handling on client applications
-

Recommendations for Improvement

1. **Return Correct Status Code:**
 - Use **400 Bad Request** for invalid input
 - Or **422 Unprocessable Entity** for validation errors
2. **Improve Error Message:**
Replace with a user-friendly message:
`"BVN must be exactly 11 digits"`
3. **Standardize Error Response Format:**

```
{  
  "status": "error",  
  "message": "BVN must be exactly 11 digits",  
  "code": "INVALID_BVN_LENGTH"  
}
```

4. **Enhance Validation Handling:**
 - Perform input validation before processing request
 - Map validation errors correctly to client error responses
5. **Hide Internal Implementation Details:**
 - Avoid exposing messages like `"child \"bvn\" fails"`

Severity: Major

Priority: High

Conclusion

The API fails to handle invalid BVN input correctly by returning an incorrect status code and a non-user-friendly error message. Proper validation and standardized error handling should be implemented to improve API reliability and usability.

Valid Testdata

API Test Case Documentation

Module: BVN Verification

API: Initialize BVN Consent

Test Case ID: TC01

Test Scenario

Initialize BVN Consent with valid request details but API returns **200 OK with no data in response body**.

Objective

To verify that the API returns meaningful response data when a valid request is sent, and does not return an empty or null response.

Preconditions

- A valid API token is generated and available
 - API endpoint is up and accessible
 - Required headers are configured properly
-

Test Data

Field	Value
BVN	22222222222 (Valid 11 digits)
Email	deepthimopuru12@example.com (Valid email)

Steps to Execute

1. Set HTTP method to **POST**
2. Enter the Initialize BVN Consent endpoint URL
3. Add required headers:
 - Authorization: Bearer Token
 - Content-Type: application/json
4. Add the request payload:

```
{  
  "bvn": "22222222222",  
  "email": "deepthimopuruxxx@gmail.com"  
}
```

5. Click **Send** to execute the request
-

Expected Result

- HTTP Status Code: **200 OK**
- Response Body should contain meaningful data such as:

```
{  
  "status": "success",  
  "message": "Consent initialized successfully",  
  "data": {  
    "reference_id": "<unique_id>",  
    "masked_phone": "<masked_phone>",  
    "masked_email": "<masked_email>"  
  }  
}
```

- Response should not be empty or null
 - Required fields (**status**, **message**, **data**) should be present
-

Actual Result

- HTTP Status Code: **200 OK**
 - Response Body: **Empty / No content returned**
-

Result Status

Failed

Defect Summary

The API returns a successful status code (200 OK) but does not provide any response body or data, making it impossible for clients to proceed with the workflow.

Observations

1. API returns **200 OK without response payload**, which is misleading
 2. No confirmation or reference details are provided to the client
 3. Breaks expected API contract and usability
 4. Client applications cannot proceed due to missing required data
-

Impact Analysis

- Blocks further steps in BVN consent flow
 - Causes failures in dependent systems
 - Leads to ambiguity for API consumers
 - Increases debugging effort due to lack of response information
-

Recommendations for Improvement

1. **Ensure Response Body is Always Returned:**
 - Include mandatory fields like **status**, **message**, and **data**
2. **Return Meaningful Data:**
 - Provide reference ID, masked contact details, or consent status
3. **Validate Backend Processing:**
 - Ensure service layer returns proper response after execution
4. **Avoid Empty Responses for Success Cases:**
 - If no data is available, return:

```
{  
  "status": "success",  
  "message": "No data available",  
  "data": {}  
}
```

}

5. Add Response Validation Checks:

- Prevent sending 200 OK when response body is empty

Severity: Critical

Priority: High

Conclusion

Although the API returns a success status (200 OK), the absence of response data makes the API unusable. A valid success response must always include meaningful data to support downstream processing and ensure a reliable user experience.

Empty contact field

API Test Case Documentation

Module: BVN Verification

API: Initialize BVN Consent

Test Case ID: TC05

Test Scenario

Initialize BVN Consent with an **empty email field** in the request payload.

Objective

To verify that the API properly validates the email field and returns an appropriate error response when the email is empty.

Preconditions

- A valid API token is generated and available
 - API endpoint is up and accessible
 - Required headers are configured properly
-

Test Data

Field	Value
BVN	22222222222 (Valid 11 digits)
Email	"" (Empty email)

Steps to Execute

1. Set HTTP method to **POST**
2. Enter the Initialize BVN Consent endpoint URL
3. Add required headers:
 - Authorization: Bearer Token
 - Content-Type: application/json
4. Add the request payload:

```
{  
  "bvn": "22222222222",  
  "email": ""  
}
```

5. Click **Send** to execute the request
-

Expected Result

- HTTP Status Code: **400 Bad Request** (or *422 Unprocessable Entity*)
- Response Body:

```
{  
  "status": "error",  
  "message": "Email is required"  
}
```

- Error message should be clear, concise, and user-friendly
-

Actual Result

- HTTP Status Code: **500 Internal Server Error**
- Response Body:

```
{  
  "status": "error",  
  "message": "child \"email\" fails because [\"email\" is not allowed to be empty]"  
}
```

Result Status

Failed

Defect Summary

The API returns a **500 Internal Server Error** for a client-side validation issue (empty email) and exposes a **technical validation error message** that is not user-friendly.

Observations

1. Incorrect HTTP status code (**500**) returned for invalid input
 2. Error message exposes internal validation structure (`child "email" fails`)
 3. Message is **not user-friendly or readable**
 4. Validation errors are not properly mapped to client error responses
-

Impact Analysis

- Misleads API consumers into thinking there is a server failure
 - Poor developer experience due to unclear error messaging
 - May cause incorrect error handling on client applications
-

Recommendations for Improvement

1. **Return Correct Status Code:**
 - Use **400 Bad Request** for missing/empty fields
 - Or **422 Unprocessable Entity** for validation errors
2. **Improve Error Message Readability:**
Replace with:
`"Email is required"`
3. **Standardize Error Response Format:**

```
{  
  "status": "error",  
  "message": "Email is required",  
  "code": "EMPTY_EMAIL"  
}
```

4. **Handle Validation at API Layer Properly:**
 - Catch validation errors and map them to appropriate responses
5. **Avoid Exposing Internal Validation Details:**

- Remove messages like `"child \"email\" fails"`
-

Severity: Major

Priority: High

Conclusion

The API incorrectly treats a client-side validation error as a server error and returns a technical message that is not user-friendly. Proper validation handling and standardized error responses are required to improve API usability and reliability.

Invalid Token

API Test Case Documentation

Module: BVN Verification

API: Initialize BVN Consent

Test Case ID: TC04

Test Scenario

Attempt to access the Initialize BVN Consent API using an **invalid API token**.

Objective

To verify that the API correctly handles unauthorized access and returns an appropriate **403 Forbidden** response with a clear error message.

Preconditions

- API endpoint is up and accessible
 - An **invalid or expired API token** is available
-

Test Data

Field	Value
Authorization Token	Invalid / Expired Token
BVN	22222222222 (Valid 11 digits)
Email	testuser@example.com (Valid email)

Steps to Execute

1. Set HTTP method to **POST**
2. Enter the Initialize BVN Consent endpoint URL
3. Add required headers:
 - Authorization: Bearer **InvalidToken123**
 - Content-Type: application/json
4. Add the request payload:

```
{  
  "bvn": "22222222222",  
  "email": "testuser@example.com"  
}
```

5. Click **Send** to execute the request
-

Expected Result

- HTTP Status Code: **403 Forbidden**
- Response Body:

```
{  
  "status": "error",  
  "message": "We couldn't verify your access. Please check your api key and try again"  
}
```

- Error message should clearly indicate authentication/authorization failure
-

Actual Result

- HTTP Status Code: **403 Forbidden**
- Response Body:

```
{  
  "status": "error",  
  "message": "We couldn't verify your access. Please check your api key and try again"  
}
```

Result Status

Passed

Observations

1. API correctly validates invalid token and restricts access
 2. Proper HTTP status code (**403 Forbidden**) is returned
 3. Error message is **clear, user-friendly, and actionable**
 4. No sensitive information is exposed in the response
-

Impact Analysis

- Ensures secure access control to the API
 - Prevents unauthorized usage
 - Improves developer experience with clear messaging
-

Recommendations for Improvement

1. **Maintain Consistency:**
 - Use the same error format across all authentication failures
2. **Optional Enhancement:**
 - Include an error code for better handling:

```
{  
  "status": "error",  
  "message": "We couldn't verify your access. Please check your api key and try again",  
  "code": "INVALID_API_KEY"  
}
```

3. **Logging & Monitoring:**
 - Log invalid access attempts for security tracking
-

Severity: High

Priority: High

Conclusion

The API correctly handles invalid token scenarios by returning a **403 Forbidden** status along with a clear and user-friendly error message. This ensures proper security enforcement and a good developer experience.

Module 3: Performance Metrics

Performance Analysis & Recommendations

Task 3: Performance Inference Based on API Response Times

Overview

Based on the execution of API test scenarios in Task 2, response times were analyzed across multiple endpoints under different conditions, including valid requests, validation failures, and authorization errors.

Performance Inference

1. Overall Response Behavior

The APIs demonstrated generally acceptable response times for standard requests; however, inconsistencies were observed across different scenarios. While some requests returned within an optimal range ($< 1-2$ seconds), others showed delays or improper handling despite simple validation failures.

2. Error Handling Impact on Performance

A notable observation is that **invalid input scenarios (e.g., wrong BVN length, empty email)** resulted in **500 Internal Server Errors**, which indicates that validation is occurring deeper in the processing layer rather than at the request boundary.

This not only affects correctness but can also **increase processing time unnecessarily**, as the request travels through multiple layers before failing.

3. Empty/Incomplete Responses with 200 Status

Cases where the API returned **200 OK with no response body** suggest incomplete backend processing or improper response handling. From a performance standpoint, this reflects **inefficient request lifecycle management**, where time is spent but no meaningful output is produced.

4. Authorization Handling Efficiency

The **403 Forbidden response for invalid tokens** was handled efficiently with clear messaging and appropriate timing. This indicates that authentication checks are likely implemented early in the request lifecycle, which is a good practice.

5. Inconsistency Across Scenarios

The variation in response handling (200, 400, 403, 500) for similar categories of input issues suggests **lack of standardized validation and error mapping**, which can indirectly impact performance predictability and system reliability.

Key Performance Concerns

- Unnecessary processing for invalid inputs
 - Improper error handling leading to server-side execution
 - Lack of early validation (input not rejected at boundary)
 - Inconsistent response structures affecting client-side efficiency
 - Occasional empty responses despite successful status codes
-

Recommendations for Improvement

1. Implement Early Input Validation

- Validate inputs (BVN length, email format, required fields) at the **API gateway or controller level**
 - Reject invalid requests immediately with **400/422 responses**
 - This reduces unnecessary load on business logic and downstream services
-

2. Standardize Error Handling

- Map all validation errors to **client-side status codes (400/422)** instead of 500
 - Use consistent and lightweight error response structures
 - This improves both performance and developer experience
-

3. Optimize Request Processing Flow

- Avoid passing invalid or incomplete data to deeper layers
 - Introduce **fail-fast mechanisms** to terminate invalid requests early
 - Ensure each request results in a **complete and meaningful response**
-

4. Improve Response Consistency

- Ensure all successful responses include required fields (**status, message, data**)
 - Avoid empty responses for 200 OK
 - Consistent responses reduce client-side retries and processing overhead
-

5. Enhance Monitoring & Logging

- Track response times for each endpoint
 - Monitor error rates (especially 500 errors caused by validation issues)
 - Use logs to identify bottlenecks and optimize slow components
-

6. Introduce Performance Benchmarks

- Define acceptable response time thresholds (e.g., < 1s for validation APIs)
 - Continuously measure against these benchmarks
 - Use alerts for threshold breaches
-

7. Caching & Reusability (If Applicable)

- Cache frequently accessed validation/reference data
 - Reduce repeated processing for similar requests
-

Conclusion

While the APIs show acceptable baseline performance, there are clear opportunities to improve efficiency and reliability by **shifting validation earlier, standardizing error handling, and ensuring consistent response structures**. Addressing these areas will not only enhance performance but also significantly improve scalability and overall API usability.

Module 4: Security Vulnerabilities

Task 4: API Security Vulnerabilities & Observations

Overview

During testing of the Adjutor APIs, multiple security aspects were evaluated, including authentication, validation, error handling, and request processing behavior. The findings below also incorporate response-time insights to highlight potential security risks and inefficiencies.

Security Vulnerabilities & Findings

- 1. Improper Authentication Handling**
APIs rely on token-based authentication; however, invalid requests sometimes return **500 errors instead of 401/403**, indicating weak validation and improper exception handling.
- 2. Lack of OTP / Multi-Factor Authentication**
Sensitive operations such as BVN consent initialization do not enforce OTP or secondary verification, increasing the risk of unauthorized access and identity misuse.
- 3. Exposure of Internal Validation Logic**
Error messages such as `"child 'bvn' fails because..."` expose internal validation structures, which could help attackers understand backend implementation.
- 4. Incorrect HTTP Status Code Usage**
Client-side validation issues returning **500 Internal Server Error** instead of **400/422** indicate poor error handling and increase security risk.
- 5. Delayed Input Validation**
Invalid inputs (e.g., incorrect BVN length, empty email) are processed at deeper layers rather than being rejected early, increasing attack surface.
- 6. Authorization Handling Gaps**
While invalid tokens return **403 Forbidden**, the implementation is not fully standardized, allowing potential misuse or token testing attempts.
- 7. Empty Responses with 200 Status**
APIs returning **200 OK with no data** may hide backend issues and create ambiguity, making it difficult to detect failures or malicious behavior.
- 8. No Rate Limiting / Throttling Observed**
Absence of rate limiting exposes APIs to brute-force and enumeration attacks (e.g., repeated BVN attempts).

9. **Replay Attack Risk**

No visible implementation of request-level protections such as nonces, timestamps, or unique request IDs to prevent replay attacks.

10. **Response Time Indicates Deep Processing of Invalid Requests**

Analysis shows total response time ~114 ms, with **~111 ms spent in server processing (TTFB)**.

This indicates that even invalid or malformed requests are processed extensively in the backend instead of being rejected early.

This behavior:

- Increases server load unnecessarily
- Opens doors for denial-of-service (DoS) style abuse
- Indicates lack of fail-fast validation mechanisms

Conclusion

While the APIs implement basic token-based security, they lack strong **defense-in-depth practices**, including early validation, proper error handling, OTP enforcement, and abuse prevention mechanisms. Additionally, response-time analysis highlights inefficiencies in request processing, which can lead to both **performance and security risks**. Addressing these gaps will significantly improve the overall security posture and system reliability.

Module 1: Signup, Login & API key Management

Lendsqr Adjutor API Test Script Documentation

Objective

The objective of this exercise was to identify and document the APIs supporting the following functionalities within the Adjutor platform and develop comprehensive test scenarios for validation:

- User Signup
 - User Login
 - API Key Management
-

API Discovery Approach

Since the Signup and Login APIs were not publicly available in the product documentation, the endpoints were identified using Browser Developer Tools (Network Tab) by monitoring API traffic during the following user actions:

- User Registration (Signup)
 - User Authentication (Login)
 - API Key Generation from the Dashboard
-

1. Signup API Testing

Endpoint

Method: POST

URL:

<https://vigil.lendsqr.com/pecunia/api/v2/onboard>

Purpose

This API is used to register a new organization and administrator account within the Adjutor platform.

Sample Request Parameters

Field	Description
name	User's full name
email	User's email address
password	User password
business_name	Organization name
phone_number	Contact number
locale	User locale
meta	Additional organization details

Expected Result

- User account is successfully created.
- Success message is returned.
- Organization record is generated.

Positive Test Scenarios

Test Scenario	Expected Result
Register with valid details	Account created successfully
Register new organization	Organization created successfully
All mandatory fields provided	Successful response returned

Negative Test Scenarios

Test Scenario	Expected Result
Existing email address	Appropriate validation error
Invalid email format	Email validation error
Empty password	Required field validation error
Missing mandatory fields	Validation failure
Invalid request payload	Bad Request response

Edge Case Testing

- Very long email addresses
- Special characters in organization name
- Empty request body
- Null values in optional fields
- Multiple rapid signup requests

Validation Checks

- Verify HTTP status code
 - Verify response structure
 - Verify success message
 - Verify account creation in the system
-

2. Login API Testing

Endpoint

Method: POST

URL:

<https://vigil.lendsqr.com/pecunia/api/v2/auth/admin/login>

Purpose

This API authenticates registered users and generates an access token for subsequent API requests.

Request Parameters

Field	Description
email	Registered email address
password	User password

Expected Result

- User is authenticated successfully.
- Session ID is generated.
- JWT access token is returned.

Positive Test Scenarios

Test Scenario	Expected Result
Valid credentials	Login successful
Registered user	Authentication successful
Token generation	Access token returned

Negative Test Scenarios

Test Scenario	Expected Result
Invalid password	Authentication failure
Unregistered email	User not found error
Missing credentials	Validation error
Invalid request payload	Bad Request response

Edge Case Testing

- Empty request body
- Multiple consecutive login attempts
- SQL Injection attempts
- Special characters in credentials

Validation Checks

- Verify HTTP status code
 - Verify token generation
 - Verify session ID creation
 - Verify user information returned correctly
-

3. API Key Management Testing

Endpoint

URL:

https://vigil.lendsqr.com/pecunia/api/v1/admin/adjutor/apps?org_id=1

Purpose

This API is used to create and manage application access for Adjutor services.

Request Parameters

Field	Description
name	Application name
description	Application description
services	List of enabled services

Expected Result

- Application is created successfully.
- Application identifier is generated.
- Access credentials become available.

Positive Test Scenarios

Test Scenario	Expected Result
Valid token provided	Request successful
Valid application details	Application created
Service selection valid	Services assigned correctly

Negative Test Scenarios

Test Scenario	Expected Result
Missing authentication token	Unauthorized error
Invalid token	Authentication failure
Expired token	Session expired error
Invalid request payload	Validation error

Edge Case Testing

- Multiple application creations
- Regeneration of API keys
- Validation of old key invalidation
- Concurrent requests

Validation Checks

- Verify application creation
 - Verify application status
 - Verify returned identifiers
 - Verify service assignments
-

Common Testing Activities

Functional Testing

- Validate business functionality.
- Verify successful and failed user flows.
- Validate response payloads.

Response Validation

Verify that all API responses contain:

- Status field
- Message field
- Data object
- Correct response structure

Security Testing

- Verify authentication enforcement.
- Verify authorization controls.
- Validate token expiration handling.
- Ensure unauthorized access is blocked.

Performance Testing

Validation Item	Expected Result
Response Time	Less than 3 seconds
Concurrent Requests	Handled successfully
System Availability	Stable under expected load

Error Handling Testing

- Verify appropriate error messages.
 - Verify correct HTTP status codes.
 - Verify graceful handling of invalid requests.
-

Conclusion

The Signup, Login, and API Key Management APIs were successfully identified and analyzed through network traffic monitoring. Comprehensive test coverage was developed, including functional, negative, edge case, security, and performance testing scenarios.

This testing approach ensures reliable validation of Adjutor's authentication and access management workflows while providing adequate coverage for both expected and unexpected user interactions.

Prepared By:

Deepthi Mopuru

Senior QA Automation Engineer

&